EXP: 1 **Date:-05/08/24**

STUDY OF PROLOG USING SWI PROLOG

TOPICS :

a) Basics of Turbo Prolog
b) Intro to Prolog programming
c) Running a simple program

- Prolog is a logical programming language and stands for PROgramming in LOGic • Created around 1972
- Preferred for AI programming and mainly used in such areas as: o Theorem proving, expert systems, NLP, …
- Logical programming is the use of mathematical logic for computer programming. **MENU**
- Files – Enables the user to load programs from disk, create new programs, save modified programs to disk, and to quit the program.
- Edit – Moves user control to the Editor panel.
- Run – Moves user control to the Dialog panel ; compiles the user program (if not already done so) in memory before running the program.
- Compile – Provides the user with choices on how to save the compiled version of the program.
- Options – Provides the user with choices on the type of compilation to be used. • Setup – Enables the user to change panel sizes, colors, and positions.

DIALOG

When a Prolog program is executing, output will be shown in the Dialog Panel **Message**
The Message Panel keeps the programmer up to date on processing activity. **Trace**
The Trace Panel is useful for finding problems in the programs you create.

PROLOG CLAUSES

- Any factual expression in Prolog is called a clause.
- There are two types of factual expressions: facts and rules
- There are three categories of statements in Prolog:

Facts: Those are true statements that form the basis for the knowledge base.

Rules: Similar to functions in procedural programming (C++, Java…) and has the form of if/then.

Queries: Questions that are passed to the interpreter to access the knowledge base and start the program.

WHAT IS A PROLOG PROGRAM ?

Prolog is used for solving problems that involve objects and the relationships between objects.

- A program consists of a database containing one or more facts and zero or more rules(next week).
- A fact is a relationship among a collection of objects. A fact is a one-line statement that ends with a full-stop.

parent (john,
bart). parent
(barbara,
bart). male
(john).
dog(fido). >> Fido is a dog or It is true that
fido is a dog sister(mary, joe). >> Mary is
Joe's sister.

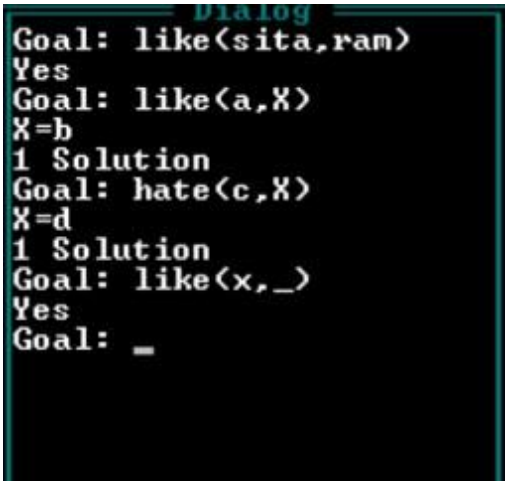
play(mary, joe, tennis). >> It is true that Mary and Joe play tennis.

- Relationships can have any number of objects.
- Choose names that are meaningful – because in Prolog names are arbitrary strings but people will have to associate meaning to them

1. Program to demonstrate a simple prolog program.

predicates
like(symbol,symbol)
hate(symbol,symbol)
clauses
like(sita,ram).
like(x,y).
like(a,b).
hate(c,d).
hate(m,n).

OUTCOME :



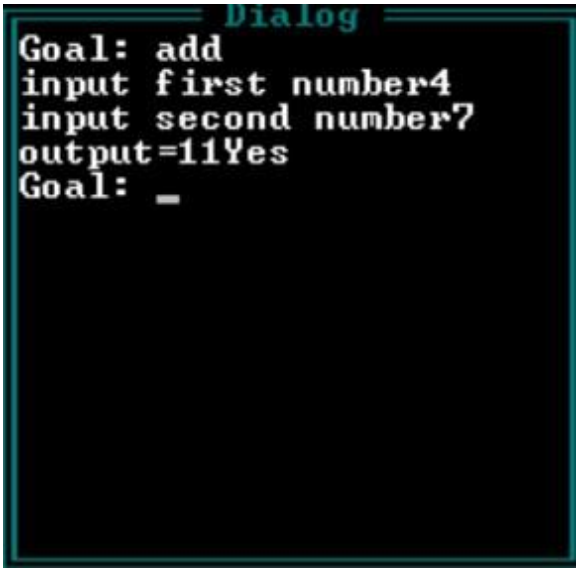
2. **Program to add two numbers.**

Predicates
add
clauses
add:-write("input first number"),

readint(X), write("input second number"),

readint(Y), Z=X+Y,write("output=",Z).

OUTCOME :

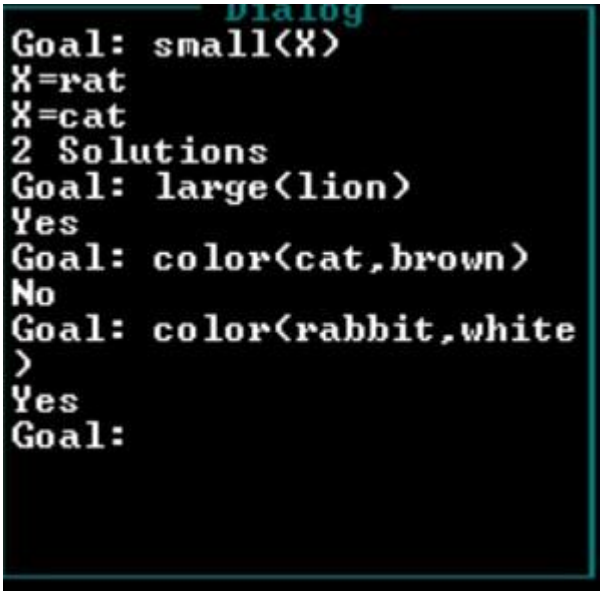


3. Program to categorise animal characteristics.

predicates
small(symbol)
large(symbol)
color(symbol,symbol)

clauses
small(rat).
small(cat).
large(lion).
color(dog,black).
color(rabbit,white).
color(X,dark):-
color(X,black);color(X,brown)

OUTCOME :



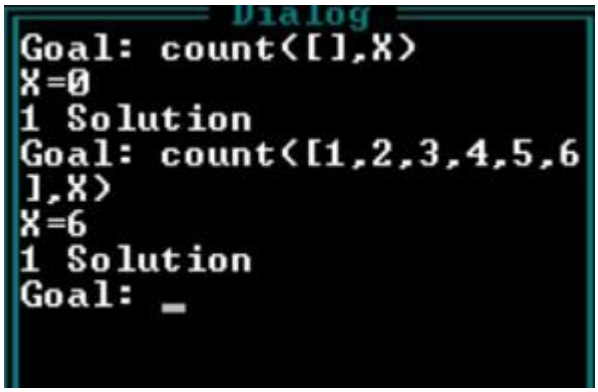
4. (a) Program to count number of elements in a list .

```
domains
x=integer
list=integer*

predicates
count(list,x)

clauses
count([],0).
count([_|T],N):-count(T,N1),N=N1+1.
```

OUTCOME :



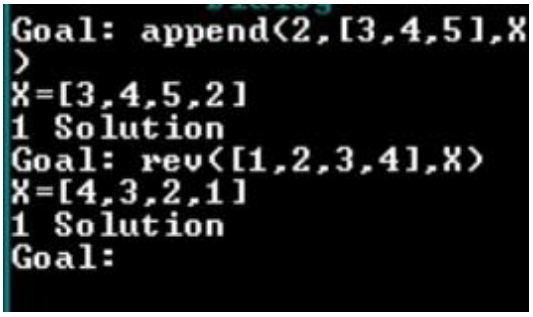
4. (b) Program to reverse the list .

domains
x=integer
list=integer*

predicates
append(x,list,list)
rev(list,list)

clauses
append(X,[],[X]).
append(X,[H|T],[H|T1]):-append(X,T,T1).
rev([],[]).
rev([H|T,rev):-rev(T,L),append(H,L,rev).

OUTCOME :



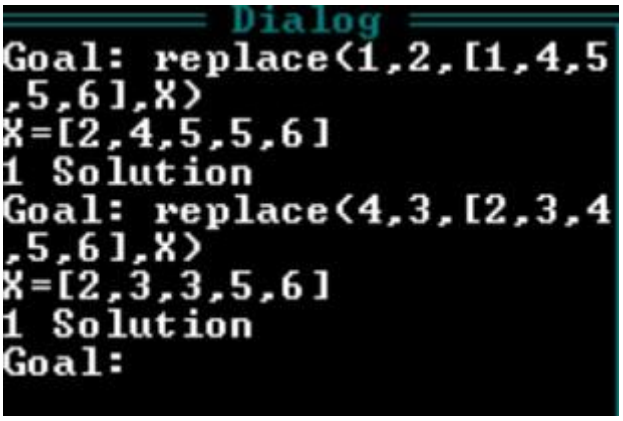
4.(C)PROGRAM TO REPLACE AN INTEGER FROM THE LIST .

domains
list=integer*

predicates
replace(integer,integer,list,list)

clauses
replace(X,Y,[X|T],[Y|T]).
replace(X,Y,[H|T],[H|T1]):-replace(X,Y,T,T1).

OUTCOME:-



4.(D)PROGRAM TO REPLACE AN INTEGER .

domains
list=integer*

predicates
del(integer,list,list)

clauses
del(X,[X|T],T).
del(X,[H|T],[H|T1]):- del(X,T,T1).

OUTCOME :

